“HOW TO DETECT A NICHE OF FRAUDSTERS?”

BY BNP PARIBAS PF

Advanced Machine Learning – Open Campus SS 2026

Alexander Neumann & Tom Hillekamp

Agenda

- Introduction
- Literature Review
- Dataset Characeristics
- Baseline Model
- Model Definition and Evaluation
- Results
- Challenges & Errors
- Discussion
- Conclusion
- Q & A

Introduction

Fraud Detection

GOAL

Find the best way to analyse shopping basket data from a retailer partner in order to detect fraud cases

Problem:

Fraudsters use **“Buy now – pay later“** but never pay

TASK

Classification – **Fraud/Not Fraud**

DATA

Tabular Data with shopping basket contents, e.g. item type, number of items, prices per item

115.988 samples:
~1.4% fraud

147 columns: 144 are grouped into 6 categories

Found on ChallengeData:
https://challengedata.ens.fr/challenges/104

Literature Review

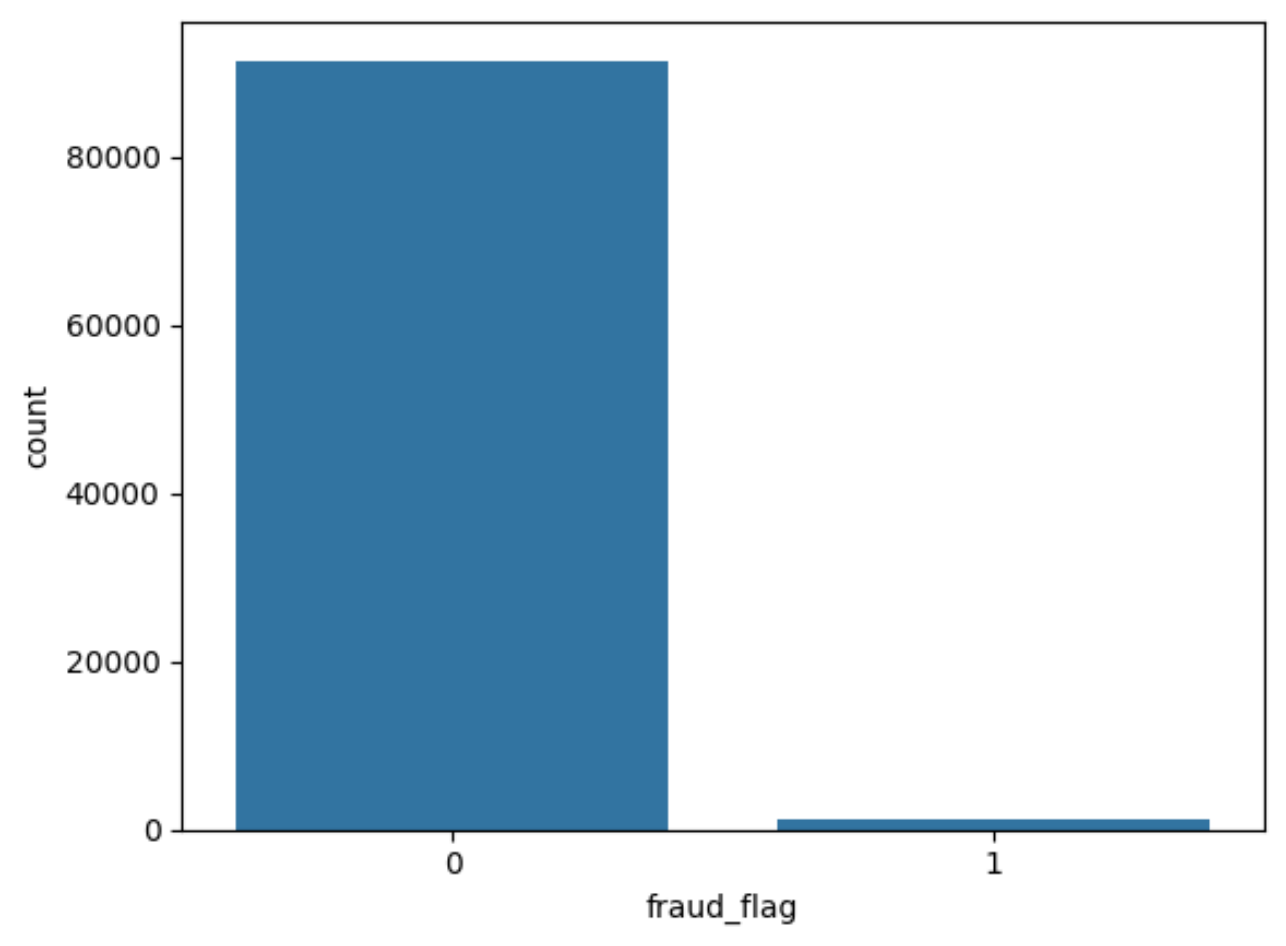
- Source 1: “A numeric-based machine learning design for detecting organized retail fraud in digital marketplaces“
  → **Rebalance data for better results, importance of proper feature choice**

- Source 2: **“**A Comprehensive Survey on Imbalanced Data Learning“
  → **Data rebalancing techniques: downsampling, SMOTE, hybrid methods, focal loss**

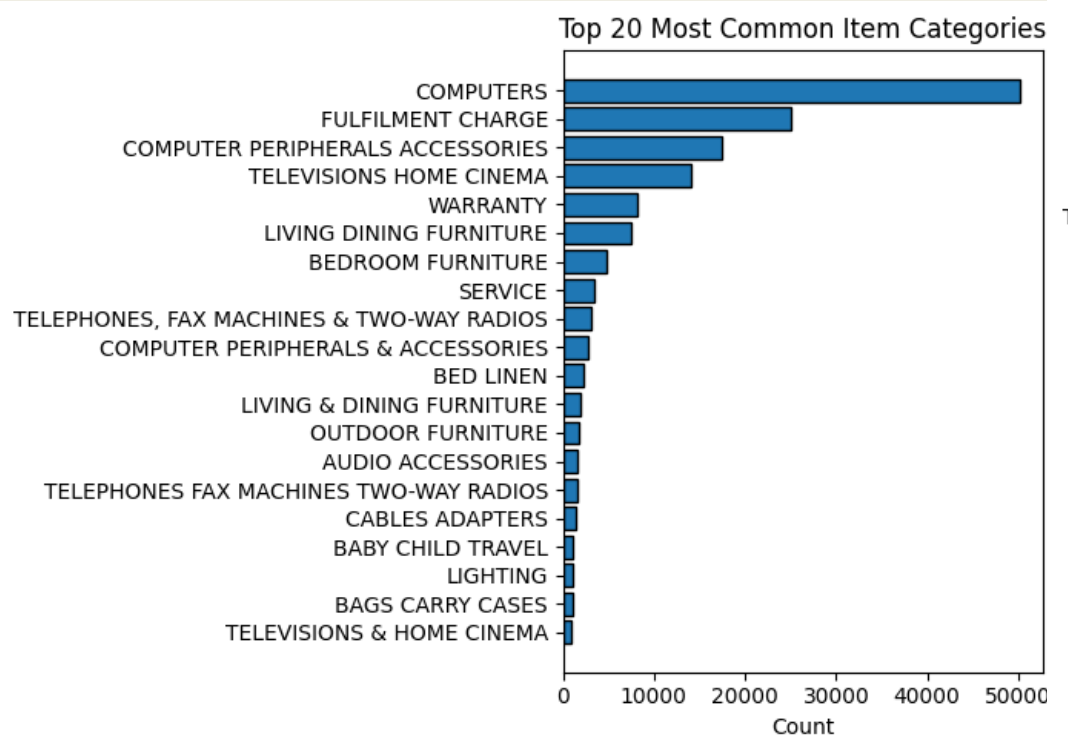
- Source 3: **“**Fraud Detection Models and their Explanations for a Buy-Now-Pay-Later Application“
  → **Comparison of techniques: Random forrest performed the best, use aggregate features**

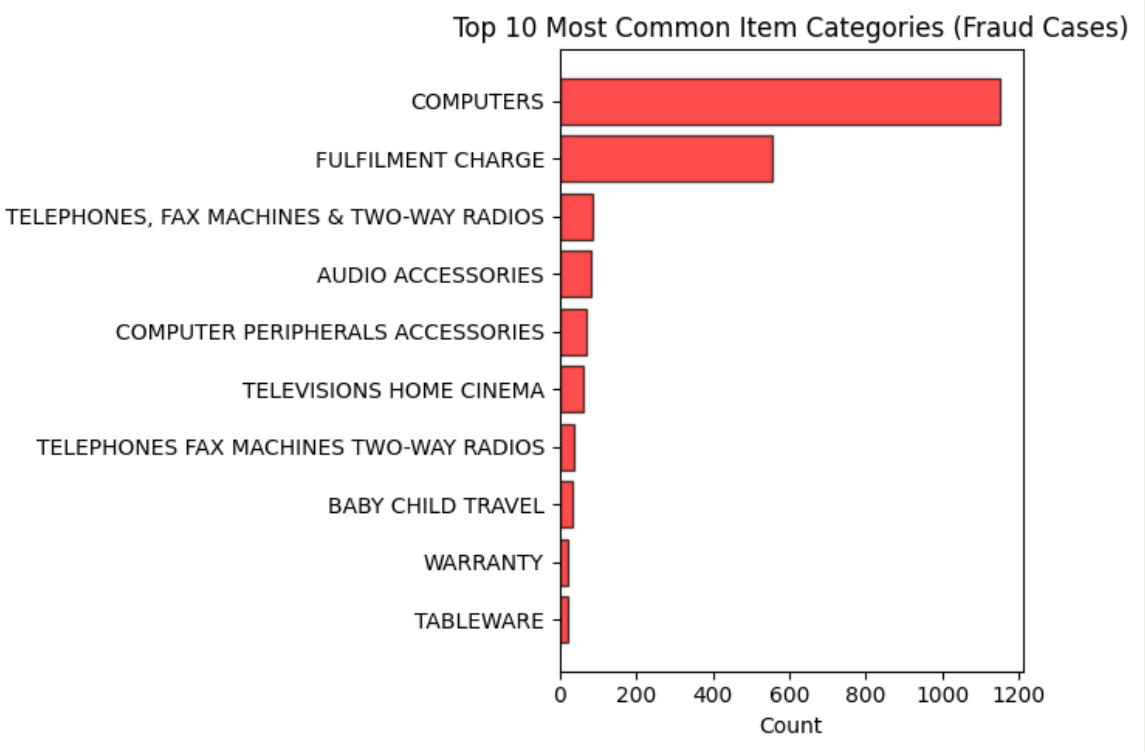
However: not much research concerning basket-level data, often includes user data etc.

Dataset Characteristics

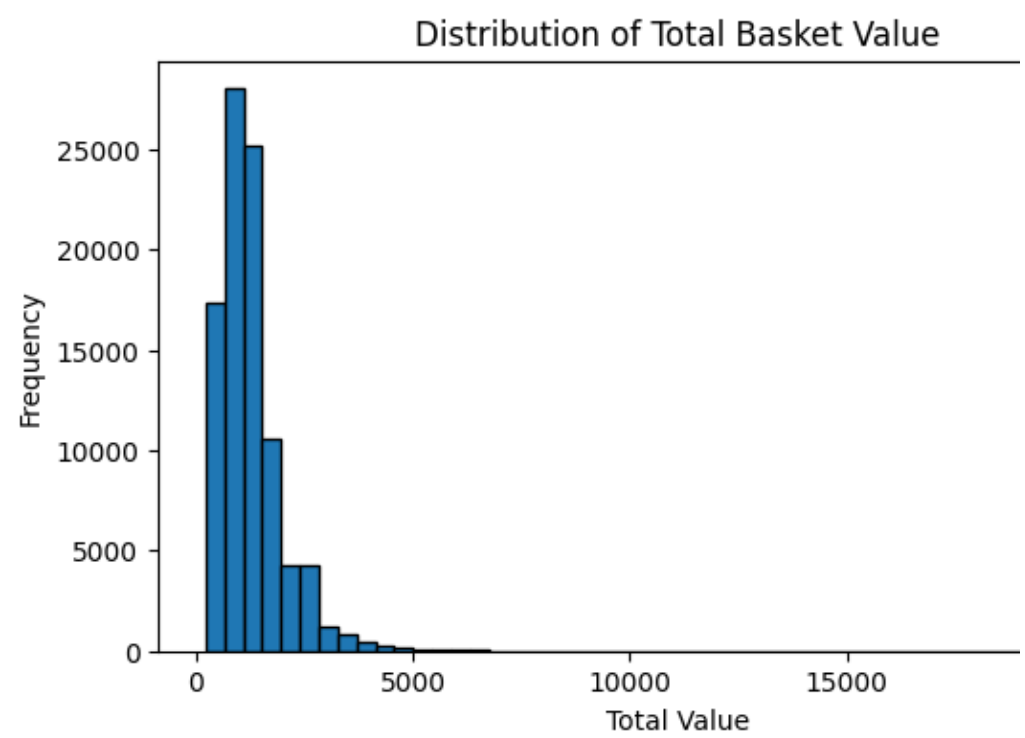


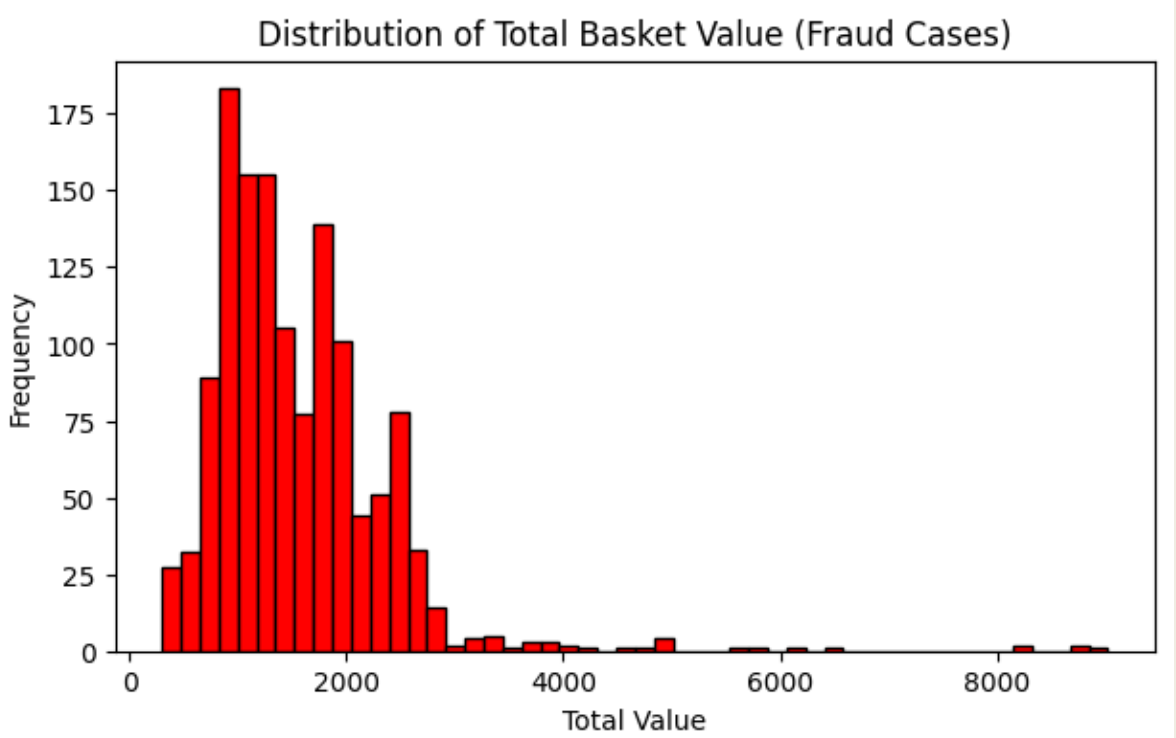
Dataset Characteristics





Dataset Characteristics





Baseline Model

- **Simple Neural Network**
- **Evaluation Metric:** PR-AUC
- **Features:** Aggregated numerical features
  - *Total number of items in basket*
  - *Total value of basket*
  - *Maximum item price*
  - *Average item price*
  - *Number of items from category “Computers“*

```
model = nn.Sequential(
            nn.Linear(N_INPUT_FEATURES, 16),
            nn.ReLU(),
            nn.Linear(16, 8),
            nn.ReLU(),
            nn.Dropout(DROPOUT_RATE),
            nn.Linear(8, 1),
        )
```

Baseline Model - Results

- **5-fold Cross Validation**: **0.0543 (+/- 0.0091) PR-AUC**
- **Test Set** (split from training data): **0.0510 PR-AUC**
- **Test Set** (public leaderboard on ChallengeData): **0.0623 PR-AUC**

→ Benchmark 1: 0,017 PR-AUC **<** **Our Baseline Model** **<** Benchmark 2: 0,1574 PR-AUC

Source: https://challengedata.ens.fr/challenges/104

Preprocessing

Feature Engineering (Aggregated Numerical Features)

```
train_x_df['total_item_count'] = train_x_df[qty_cols].sum(axis=1)
train_x_df['total_price'] = train_x_df[price_cols].sum(axis=1)
train_x_df['max_price'] = train_x_df[price_cols].max(axis=1)
train_x_df['mean_price'] = train_x_df[price_cols].sum(axis=1) / train_x_df['total_item_count']
```

Categorical Feature Encoding (Example)

Categorical Feature	Numerical Vocabulary
2HP ELITEBOOK 850V6	1
2LOGITECH PEBBLE M350 BLUETOOTH MOUSE	2

The numerically-encoded categorical data can be fed into the model's embedding layers

Models to compare

1. Embeddings Model

```
FraudModel(
  (item_embedding): Embedding(178, 8, padding_idx=0)
  (make_embedding): Embedding(888, 8, padding_idx=0)
  (goods_embedding): Embedding(17029, 103, padding_idx=0)
  (fc): Sequential(
    (0): Linear(in_features=171, out_features=64, bias=True)
    (1): ReLU()
    (2): Dropout(p=0.25, inplace=False)
    (3): Linear(in_features=64, out_features=32, bias=True)
    (4): ReLU()
    (5): Dropout(p=0.2, inplace=False)
    (6): Linear(in_features=32, out_features=1, bias=True)
  )
)
```

2. XGBoost

```
xgb = XGBClassifier(
    n_estimators=3000,
    learning_rate=0.02,
    max_depth=8,
    min_child_weight=5,
    scale_pos_weight=scale,
    subsample=0.8,
    colsample_bytree=0.6,
    gamma=1,
    eval_metric='aucpr',
    early_stopping_rounds=100,
    random_state=42,
)
```

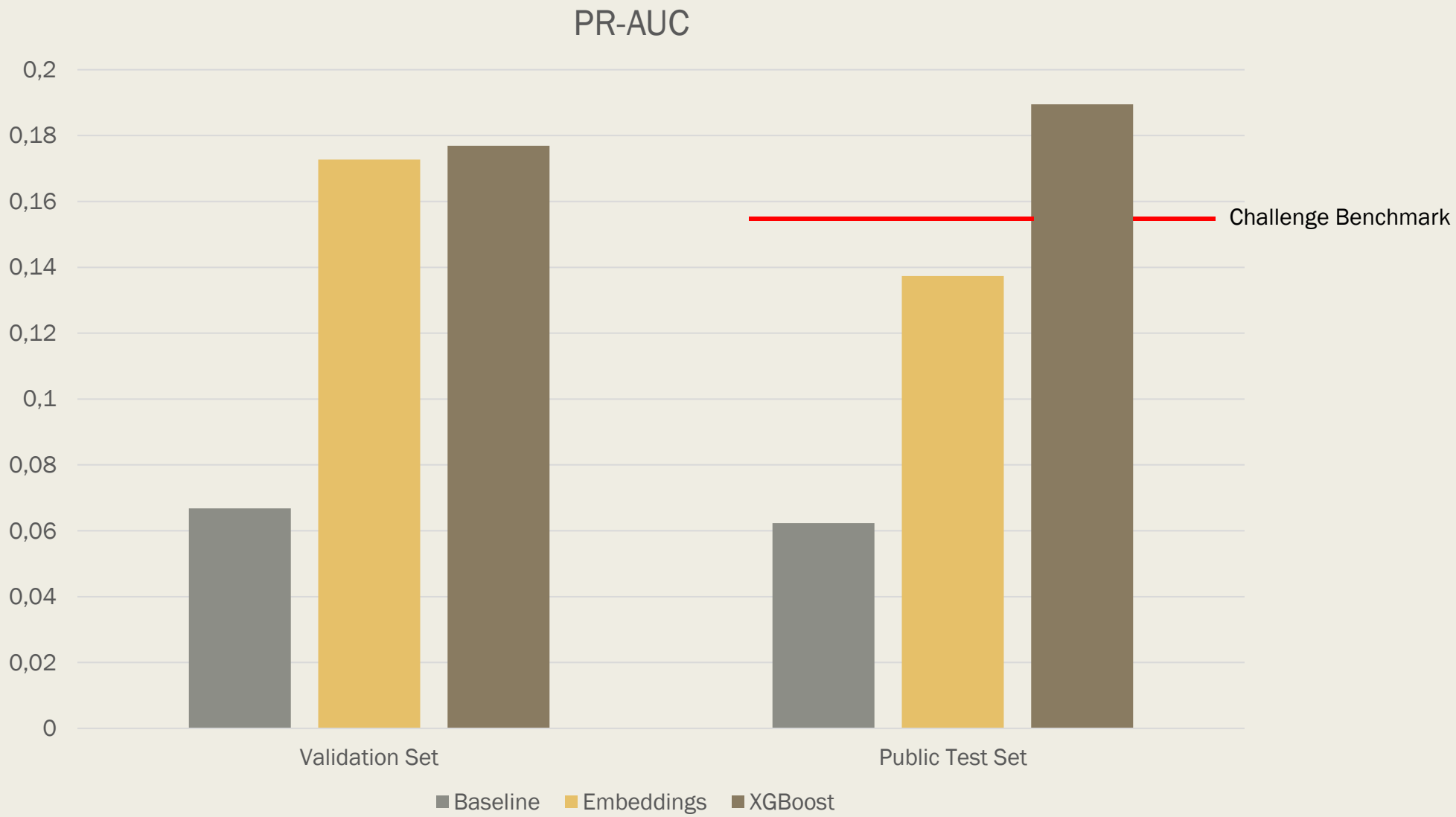
ANN with Embeddings

```
class FraudModel(nn.Module):
    def __init__(self, ...):
        super().__init__()

        # Shared embedding tables
        self.item_embedding = nn.Embedding(embedding_sizes["item"],  item_dim, padding_idx=0)
        self.make_embedding = nn.Embedding(embedding_sizes["make"], make_dim, padding_idx=0)
        self.goods_embedding = nn.Embedding(embedding_sizes["goods"], goods_dim, padding_idx=0)
        total_embedding_dim = (item_dim + make_dim + goods_dim)

        # Main classifier
        layers = []
        in_dim = total_embedding_dim + n_numeric

        for out_dim, dropout in zip(fc_layers, dropout_rates):
            layers += [nn.Linear(in_dim, out_dim), nn.ReLU(), nn.Dropout(dropout),]
            in_dim = out_dim

        layers.append(nn.Linear(in_dim, 1))
        self.fc = nn.Sequential(*layers)
```

ANN with Embeddings

```
    def masked_mean_pooling(self, embeddings, x):
        [...]

    def forward(self, x_cat, x_num):
        # Split categorical groups
        item_x = x_cat[:, 0:24]
        make_x = x_cat[:, 24:48]
        goods_x = x_cat[:, 48:72]

        # Embeddings
        item_emb = self.item_embedding(item_x)
        make_emb = self.make_embedding(make_x)
        goods_emb = self.goods_embedding(goods_x)

        # Masked mean pooling
        item_pooled = self.masked_mean_pooling(item_emb, item_x)
        make_pooled = self.masked_mean_pooling(make_emb, make_x)
        goods_pooled = self.masked_mean_pooling(goods_emb, goods_x)

        # Concatenate pooled embeddings
        x = torch.cat([item_pooled, make_pooled, goods_pooled, x_num], dim=1)
        # MLP
        x = self.fc(x)

        return x.squeeze()
```

Results


PR-AUC
0,2
0,18
0,16
0,14
0,12
0,1
0,08
0,06
0,04
0,02
0
Challenge Benchmark
Validation Set
Public Test Set
Baseline
Embeddings
XGBoost

Results

99	May 19, 2026, 3:22 p.m.	ridahmed	0.1922
100	Jan. 22, 2024, 10:54 p.m.	Kzj	0.1915
101	Nov. 10, 2025, 10:28 a.m.	TNNDD2	0.1911
102	**June 7, 2026, 6 p.m.**	**tomhill**	**0.1895**
103	Nov. 27, 2023, 4:22 p.m.	BOUTAUD_FAUCHEUX	0.1892
104	May 19, 2026, 3:57 p.m.	hopalala	0.1883
105	Nov. 23, 2023, 8:14 a.m.	ZABLIT_MICHELON	0.1877

Challenges and Errors

- How to deal with the **categorical** columns in the dataset?
  - *Vocabularies + Embeddings*
- How to deal with the **varying number** of items per basket?
  - *Masked Mean Pooling*
- What **parameters** to **optimize** during an Optuna study?
  - *Trial & Error, Minimizing Parameter Space*

- **Improving results** of embedding model
  - *Tried Oversampling/Undersampling → no improvements*
  - *Hyperparameter Optimization → only small improvements*

  → XGBoost
- **Ensemble Approach** (ANN + XGBoost) → no improvements

Discussion

- Embeddings significantly improved the results compared to Baseline
  - *0.06 to 0.17 PR-AUC*
- Tree-based model performed even better than embedding model
  - *Overfitting of Embedding Model? (Drop in PR-AUC on Test Set)*

Conclusion and Future Work

- We were able to train a model that performs better than the Benchmark on the Public Test Set
  - *Rank 102: 0.1895 PR-AUC (XGBoost)*
  - *Rank 225: Benchmark*

Future Work:

- Try other tree-based algorithms
- Refine embedding approach
  - *Maybe transfer learning?*

Thank you! ☺

Q & A